

TypeScript

From Zero to Associate Software Engineer

A Complete Full-Stack Learning & Interview Preparation Guide

L1 & L2 Ready

Covers TypeScript fundamentals, advanced types, generics, OOP, Node.js, Express, React, databases, testing, and 100+ interview questions.

Table of Contents

Part I —

Foundations

- Chapter 1** Introduction to TypeScript
- Chapter 2** Setting Up Your TypeScript Environment
- Chapter 3** Basic Types and Type Annotations
- Chapter 4** Functions in TypeScript
- Chapter 5** Objects, Interfaces, and Type Aliases
- Chapter 6** Arrays, Tuples, and Enums
- Chapter 7** Union, Intersection, and Literal Types
- Chapter 8** Type Narrowing and Type Guards

Part II —

Intermediate

- Chapter 9** Classes and Object-Oriented Programming
- Chapter 10** Generics
- Chapter 11** Utility Types
- Chapter 12** Modules, Namespaces, and tsconfig

Part III —

Advanced

- Chapter 13** Advanced Type System (keyof, typeof, mapped, conditional)
- Chapter 14** Decorators and Metadata
- Chapter 15** Async/Await and Promise Typing

Part IV — Full

Stack

TypeScript

- Chapter 16** TypeScript with Node.js and Express
- Chapter 17** TypeScript with React
- Chapter 18** TypeScript with Databases (Prisma / TypeORM)
- Chapter 19** Error Handling, Testing, and Best Practices

Part V —
**Interview
Preparation**

Chapter 20	L1 Interview Questions (Entry-Level)
Chapter 21	L2 Interview Questions (Associate Engineer)
Chapter 22	Coding Challenges with Solutions
Chapter 23	Behavioral Questions & Final Tips
Appendix	Resources, Cheat Sheets, and Roadmap

Part I — Foundations

Chapter 1: Introduction to TypeScript

What is TypeScript?

TypeScript is a strongly-typed, open-source programming language developed and maintained by Microsoft. It is a strict syntactical superset of JavaScript — every valid JavaScript program is also a valid TypeScript program. TypeScript adds **optional static typing**, **classes**, **interfaces**, and modern language features that transpile down to plain JavaScript that runs anywhere JavaScript runs: browsers, Node.js, Deno, Bun, and serverless platforms.

Why TypeScript?

- **Catch bugs at compile time** — type errors are surfaced before code runs.
- **Better tooling** — IDE autocompletion, refactoring, and inline documentation.
- **Scalability** — large codebases stay maintainable as teams grow.
- **Self-documenting code** — function signatures express intent explicitly.
- **Industry standard** — used at Microsoft, Google, Airbnb, Slack, Shopify, and most modern startups.

TypeScript vs JavaScript

```
// JavaScript – runs, but breaks at runtime
function add(a, b) {
  return a + b;
}
add('5', 10); // '510' – silent string concatenation bug

// TypeScript – refuses to compile
function add(a: number, b: number): number {
  return a + b;
}
add('5', 10);
// Error: Argument of type 'string' is not assignable to parameter of type 'number'.
```

How TypeScript Works

TypeScript code lives in **.ts** or **.tsx** files. The TypeScript compiler (**tsc**) reads your code, performs type-checking, and emits clean JavaScript. Crucially, **all type information is erased** at compile time — there are no types in the output JavaScript. This is called **type erasure** and it is a frequent interview question.

***Note:** TypeScript types exist only during development. At runtime, your code is plain JavaScript. This means you cannot use TypeScript types to validate user input at runtime — use a library like Zod or Yup for that.*

Chapter 2: Setting Up Your TypeScript Environment

Installation

```
# Install Node.js first (https://nodejs.org)
# Then install TypeScript globally
npm install -g typescript

# Verify
tsc --version

# Or install per project (recommended)
npm init -y
npm install --save-dev typescript @types/node ts-node
```

Your First TypeScript File

```
// hello.ts
const message: string = 'Hello, TypeScript!';
console.log(message);

// Compile
// tsc hello.ts -> produces hello.js

// Or run directly with ts-node
// npx ts-node hello.ts
```

Initializing tsconfig.json

Every TypeScript project needs a **tsconfig.json** at its root. This file tells the compiler how to behave.

```
tsc --init # generates a default tsconfig.json
```

A Sensible tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",           // JS version to emit
    "module": "commonjs",        // module system (use "esnext" for ESM)
    "lib": ["ES2022", "DOM"],     // available built-in APIs
    "outDir": "./dist",          // output folder
    "rootDir": "./src",          // source folder
    "strict": true,              // enables ALL strict checks (essential!)
    "esModuleInterop": true,     // smoother CJS/ESM interop
    "skipLibCheck": true,        // faster compiles
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,   // import .json files
    "sourceMap": true,           // debug TS in browser/node
    "declaration": true          // emit .d.ts type files
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

Key Strict Flags (Interview Favorite)

- **noImplicitAny** — variables without inferable types must be annotated.
- **strictNullChecks** — null and undefined are not assignable to other types.
- **strictFunctionTypes** — function parameter types are checked bivariantly disabled.
- **strictPropertyInitialization** — class fields must be initialized.

- **noImplicitThis** — **this** must have an explicit type.
- **alwaysStrict** — emits 'use strict' in output JS.

Useful Scripts (package.json)

```
{
  "scripts": {
    "build": "tsc",
    "dev": "ts-node src/index.ts",
    "watch": "tsc --watch",
    "start": "node dist/index.js",
    "type-check": "tsc --noEmit"
  }
}
```

Chapter 3: Basic Types and Type Annotations

Primitive Types

TypeScript supports JavaScript's seven primitives, each with a corresponding type:

```
// number – all numbers, integer or float
let age: number = 30;
let price: number = 9.99;
let hex: number = 0xff;

// string
let name: string = 'Ada';
let greeting: string = `Hello, ${name}`;

// boolean
let isActive: boolean = true;

// bigint – for integers larger than Number.MAX_SAFE_INTEGER
let big: bigint = 9007199254740993n;

// symbol – unique identifier
let id: symbol = Symbol('id');

// null and undefined
let n: null = null;
let u: undefined = undefined;
```

Type Inference

TypeScript is smart — you don't need to annotate everything. The compiler infers types from initial values. Idiomatic TypeScript leans on inference.

```
let count = 10;           // inferred as number
let title = 'Engineer';   // inferred as string
let ready = false;        // inferred as boolean

count = 'twenty';         // Error – Type 'string' is not assignable to type 'number'.
```

Special Types: any, unknown, never, void

any — opt out of type checking. Avoid it; it defeats the purpose of TypeScript.

```
let anything: any = 5;
anything = 'now a string';
anything.foo.bar.baz(); // No error – and likely crashes at runtime.
```

unknown — the type-safe alternative to **any**. You must narrow it before use.

```
let value: unknown = 'hello';
// value.toUpperCase(); // Error – Object is of type 'unknown'.

if (typeof value === 'string') {
  console.log(value.toUpperCase()); // OK – narrowed to string
}
```

void — represents the absence of a return value (functions that return nothing).

```
function logMessage(msg: string): void {
  console.log(msg);
  // no return
}
```

never — represents a value that *can never occur*. Used for functions that throw or loop forever, and for exhaustive switch statements.

```
function throwError(msg: string): never {
  throw new Error(msg);
}

function infinite(): never {
  while (true) { /* ... */ }
}
```

any vs unknown — Interview Killer

This is one of the most common L1/L2 questions. Both can hold any value, but:

- **any** disables type checking — you can do anything with it, including bug-creating things.
- **unknown** forces you to narrow (check the type) before doing anything with it.
- Rule of thumb: prefer **unknown** for incoming data of unknown shape (API responses, JSON.parse, etc.).

Type Assertions (Casting)

```
// Use 'as' to tell the compiler 'trust me, I know better'
const input = document.getElementById('name') as HTMLInputElement;
input.value = 'Ada';

// Alternative angle-bracket syntax (not allowed in .tsx files)
const input2 = <HTMLInputElement>document.getElementById('name');

// Double assertion (when types don't overlap) – use sparingly!
const num = ('5' as unknown) as number;
```

Note: Type assertions do NOT change the runtime value. They only tell the compiler to treat a value as a different type. If you're wrong, you'll get a runtime error.

Chapter 4: Functions in TypeScript

Function Declarations with Types

```
// Parameter types and return type
function multiply(a: number, b: number): number {
    return a * b;
}

// Arrow function
const divide = (a: number, b: number): number => a / b;

// Return type can be inferred (often preferred for simple cases)
const square = (n: number) => n * n; // inferred: number
```

Optional and Default Parameters

```
// Optional with ?
function greet(name: string, title?: string): string {
    return title ? `${title} ${name}` : name;
}
greet('Ada'); // 'Ada'
greet('Ada', 'Dr. '); // 'Dr. Ada'

// Default values (parameter is automatically optional)
function buildUrl(path: string, host: string = 'localhost'): string {
    return `https://${host}${path}`;
}
```

Note: Optional parameters must come AFTER required parameters. Default values implicitly make parameters optional.

Rest Parameters

```
function sum(...nums: number[]): number {
    return nums.reduce((acc, n) => acc + n, 0);
}
sum(1, 2, 3, 4); // 10
```

Function Type Expressions

```
// Describe a function's shape
type BinaryOp = (a: number, b: number) => number;

const add: BinaryOp = (x, y) => x + y;
const sub: BinaryOp = (x, y) => x - y;

// Useful for callbacks
function apply(fn: (n: number) => number, value: number): number {
    return fn(value);
}
apply(n => n * 2, 5); // 10
```

Function Overloading

Overloading lets a single function accept different argument shapes. Declare multiple signatures, then a single implementation.

```
// Overload signatures
function format(value: string): string;
function format(value: number): string;
function format(value: Date): string;
// Implementation signature (not callable directly)
```

```
function format(value: string | number | Date): string {
    if (value instanceof Date) return value.toISOString();
    return String(value);
}

format('hi');           // OK
format(42);             // OK
format(new Date());     // OK
```

void vs undefined Return

```
function logA(msg: string): void { console.log(msg); }
function logB(msg: string): undefined { console.log(msg); return undefined; }

// void is more permissive in callback positions – callers may return anything,
// but the value is ignored. This is why Array#forEach uses void callbacks.
[1,2,3].forEach((n): void => n * 2); // ignored, but OK
```

this Parameter

```
interface Card { suit: string; value: number; }

function describe(this: Card): string {
    return `${this.value} of ${this.suit}`;
}

// 'this' is a fake first parameter – TypeScript checks the binding context.
```

Chapter 5: Objects, Interfaces, and Type Aliases

Object Type Annotations

```
// Inline object type
function printUser(user: { name: string; age: number }): void {
    console.log(`${user.name}, ${user.age}`);
}

// Optional properties
function printUser2(user: { name: string; age?: number }): void {
    console.log(user.age ?? 'unknown age');
}
```

Type Aliases

```
type User = {
    id: number;
    name: string;
    email: string;
    isAdmin?: boolean; // optional
    readonly createdAt: Date; // immutable
};

const u: User = { id: 1, name: 'Ada', email: 'a@b.com', createdAt: new Date() };
// u.createdAt = new Date(); // Error – readonly
```

Interfaces

```
interface User {
    id: number;
    name: string;
    email: string;
}

interface Admin extends User { // extends inheritance
    permissions: string[];
}

const a: Admin = {
    id: 1, name: 'Ada', email: 'a@b.com',
    permissions: ['read', 'write']
};
```

Interface vs Type — Classic Interview Question

Both define object shapes and are largely interchangeable. Key differences:

- **Interface** — can be re-opened (declaration merging). Ideal for library/public API types.
- **Type** — cannot be re-opened. Supports unions, intersections, mapped types, conditional types — everything an interface cannot do.
- **Interface** — uses **extends** for inheritance. **Type** uses **&** intersection.
- Best practice: use **interface** for object shapes; use **type** for unions, primitives, tuples, and computed types.

```
// Declaration merging – only interfaces can do this
interface Window { myCustomProp: string }
interface Window { anotherProp: number }
// Window now has BOTH properties.

// Union – only type aliases can do this
```

```
type Status = 'idle' | 'loading' | 'done';
```

Index Signatures

```
// When keys are dynamic
interface StringMap {
  [key: string]: string;
}

const headers: StringMap = {
  'content-type': 'application/json',
  'x-api-key': 'abc123'
};

// Mixed signatures: known + dynamic keys
interface ApiResponse {
  status: number;           // known
  [extra: string]: any;     // any additional dynamic keys
}
```

Excess Property Checks

```
interface Point { x: number; y: number }

// Object literal – checked strictly
const p1: Point = { x: 1, y: 2, z: 3 }; // Error: 'z' does not exist

// Through a variable – not checked (structural typing)
const tmp = { x: 1, y: 2, z: 3 };
const p2: Point = tmp; // OK
```

Note: TypeScript uses structural typing (a.k.a. duck typing): two types are compatible if their members are compatible, regardless of the type name. Excess property checks are a stricter check applied only to fresh object literals.

Chapter 6: Arrays, Tuples, and Enums

Arrays

```
// Two equivalent syntaxes
let scores: number[] = [90, 85, 100];
let names: Array<string> = ['Ada', 'Linus'];

// Array of objects
let users: { id: number; name: string }[] = [
  { id: 1, name: 'Ada' },
  { id: 2, name: 'Linus' }
];

// Multi-dimensional
let matrix: number[][] = [[1,2],[3,4]];

// readonly array
const fixed: readonly number[] = [1, 2, 3];
// fixed.push(4); // Error
```

Tuples

Tuples are fixed-length arrays where each position has its own type. Great for returning multiple values from a function (think React's `useState`).

```
// Position matters
let pair: [string, number] = ['age', 30];
// pair = [30, 'age']; // Error

// Useful pattern: useState-like return
function counter(): [number, () => void] {
  let n = 0;
  return [n, () => { n++; }];
}
const [count, increment] = counter();

// Named tuple (TypeScript 4.0+) – improves docs
type HttpResponse = [status: number, body: string];

// Optional tuple elements
type Args = [string, number?];

// Rest in tuples
type StringNumberBooleans = [string, number, ...boolean[]];
const ex: StringNumberBooleans = ['hi', 1, true, false, true];
```

Enums

Enums define a set of named constants. TypeScript supports numeric and string enums.

```
// Numeric enum – values auto-increment from 0
enum Direction { Up, Down, Left, Right }
// Direction.Up === 0, Direction.Down === 1, ...

// Start from a custom value
enum HttpStatus { OK = 200, NotFound = 404, ServerError = 500 }

// String enum – must explicitly assign
enum LogLevel {
  Debug = 'DEBUG',
  Info = 'INFO',
  Warn = 'WARN',
}
```

```
    Error = 'ERROR'
  }

function log(level: LogLevel, msg: string) { /* ... */ }
log(LogLevel.Info, 'Server started');
```

Const Enums and Alternatives

Regular enums generate runtime objects. **const enum** inlines values for performance but has caveats. Many modern teams prefer plain unions of string literals instead.

```
// const enum – inlined, no runtime object emitted
const enum Color { Red = 'red', Green = 'green', Blue = 'blue' }
const c = Color.Red; // compiles to: const c = 'red';

// Modern alternative – union of literals
type Color = 'red' | 'green' | 'blue';
const c2: Color = 'red';

// Or 'as const' object
const Status = {
  Idle: 'idle',
  Loading: 'loading',
  Done: 'done'
} as const;
type Status = typeof Status[keyof typeof Status];
```

Chapter 7: Union, Intersection, and Literal Types

Union Types (|)

A value can be one of several types.

```
type ID = string | number;

function printId(id: ID): void {
  // Only methods common to BOTH types are accessible
  console.log(id); // OK – both have toString
  // id.toUpperCase(); // Error – number doesn't have it

  // Narrow with typeof
  if (typeof id === 'string') {
    console.log(id.toUpperCase());
  } else {
    console.log(id.toFixed(0));
  }
}
```

Intersection Types (&)

Combine multiple types into one. Result has all properties.

```
interface HasName { name: string }
interface HasAge { age: number }

type Person = HasName & HasAge;

const p: Person = { name: 'Ada', age: 30 };
```

Literal Types

A literal type is a more specific subtype of a primitive — a single exact value. Combined with unions, they create powerful, expressive APIs.

```
// String literals
type Direction = 'north' | 'south' | 'east' | 'west';
let dir: Direction = 'north';
// dir = 'up'; // Error

// Numeric literals
type DiceRoll = 1 | 2 | 3 | 4 | 5 | 6;

// Boolean literal
type True = true;

// Mixed
type Result = { ok: true; data: string } | { ok: false; error: string };
```

Discriminated (Tagged) Unions

A foundational pattern. Use a common literal property (the 'discriminant') so TypeScript can narrow within the union. This pattern dominates production codebases and shows up in many L2 interviews.

```
type Shape =
  | { kind: 'circle'; radius: number }
  | { kind: 'square'; side: number }
  | { kind: 'rectangle'; width: number; height: number };

function area(s: Shape): number {
  switch (s.kind) {
```

```
    case 'circle':    return Math.PI * s.radius ** 2;
    case 'square':    return s.side ** 2;
    case 'rectangle': return s.width * s.height;
    default:
        // exhaustive check – see Chapter 8
        const _exhaustive: never = s;
        return _exhaustive;
    }
}
```


Chapter 8: Type Narrowing and Type Guards

Narrowing is how TypeScript moves from a broad type (e.g. **string | number**) to a specific one (just **string**) within a code block. Master this and you've cleared a huge L1/L2 conceptual hurdle.

typeof Guards

```
function pad(value: string | number, length: number): string {
  if (typeof value === 'number') {
    return ' '.repeat(length) + value.toFixed(2);
  }
  return value.padStart(length, ' ');
}
```

Truthiness Narrowing

```
function printName(name: string | null | undefined) {
  if (name) {
    console.log(name.toUpperCase()); // narrowed to string
  }
}
```

Equality Narrowing

```
function compare(a: string | number, b: string | boolean) {
  if (a === b) {
    // Both must be strings (the only overlapping type)
    console.log(a.toUpperCase(), b.toUpperCase());
  }
}
```

instanceof Guards

```
function logDate(d: Date | string) {
  if (d instanceof Date) {
    console.log(d.toISOString());
  } else {
    console.log(d);
  }
}
```

in Operator Guards

```
type Fish = { swim: () => void };
type Bird = { fly: () => void };

function move(a: Fish | Bird) {
  if ('swim' in a) {
    a.swim();
  } else {
    a.fly();
  }
}
```

User-Defined Type Guards

When the built-in checks aren't enough, write your own. The return type **param is Type** tells TypeScript to narrow on success.

```
interface User { id: number; name: string }

function isUser(value: unknown): value is User {
  return (
```

```

    typeof value === 'object' &&
    value !== null &&
    'id' in value &&
    'name' in value
  );
}

const data: unknown = await fetch('/me').then(r => r.json());
if (isUser(data)) {
  console.log(data.name); // safe – narrowed to User
}

```

Assertion Functions

```

function assertIsString(value: unknown): asserts value is string {
  if (typeof value !== 'string') throw new Error('Not a string!');
}

function processInput(input: unknown) {
  assertIsString(input);
  // From this point on, TypeScript knows input is a string
  console.log(input.toUpperCase());
}

```

Exhaustiveness Checking with never

```

type Action =
  | { type: 'login'; user: string }
  | { type: 'logout' }
  | { type: 'fetch'; id: number };

function handle(a: Action): void {
  switch (a.type) {
    case 'login': return console.log('Hi', a.user);
    case 'logout': return console.log('Bye');
    case 'fetch': return console.log('Fetching', a.id);
    default:
      const _exhaustive: never = a; // compile error if Action grows
      return _exhaustive;
  }
}

```

Note: Exhaustiveness checking is a key safety pattern. If a new variant is added to the union and the switch is not updated, the compiler errors at the 'never' assignment. This is a frequent topic in L2 system design discussions.

Part II — Intermediate TypeScript

Chapter 9: Classes and Object-Oriented Programming

Basic Class

```
class Person {
  name: string;
  age: number;

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }

  greet(): string {
    return `Hi, I'm ${this.name}`;
  }
}

const p = new Person('Ada', 30);
console.log(p.greet());
```

Access Modifiers

- **public** — accessible everywhere (default).
- **private** — accessible only inside the class.
- **protected** — accessible in the class and its subclasses.
- **readonly** — can be set in constructor, never reassigned afterwards.

```
class Account {
  public readonly id: string;
  private balance: number;
  protected owner: string;

  constructor(id: string, owner: string, initial: number) {
    this.id = id;
    this.owner = owner;
    this.balance = initial;
  }

  deposit(n: number): void { this.balance += n; }
  getBalance(): number { return this.balance; }
}
```

Parameter Property Shorthand

```
// Verbose
class A {
  private name: string;
  constructor(name: string) { this.name = name; }
}

// Shorthand – declare and assign in one step
class B {
  constructor(private name: string, public readonly id: number) {}
}
```

Inheritance

```

class Animal {
  constructor(public name: string) {}
  move(): void { console.log(`${this.name} moves.`); }
}

class Dog extends Animal {
  constructor(name: string, public breed: string) {
    super(name); // call parent constructor
  }
  bark(): void { console.log('Woof!'); }
  // override
  move(): void {
    super.move();
    console.log('wagging tail');
  }
}

```

Abstract Classes

Abstract classes cannot be instantiated directly. They define a contract and may provide partial implementation.

```

abstract class Shape {
  abstract area(): number; // must be implemented
  describe(): string { // concrete method
    return `Area: ${this.area()}`;
  }
}

class Circle extends Shape {
  constructor(public r: number) { super(); }
  area(): number { return Math.PI * this.r ** 2; }
}

// new Shape(); // Error – cannot instantiate abstract class

```

Interface Implementation

```

interface Logger {
  log(msg: string): void;
  warn(msg: string): void;
}

class ConsoleLogger implements Logger {
  log(msg: string) { console.log(msg); }
  warn(msg: string) { console.warn(msg); }
}

// Implementing multiple interfaces
interface Serializable { toJSON(): string }
class FileLogger implements Logger, Serializable {
  log(msg: string) { /* write to file */ }
  warn(msg: string) { /* write to file */ }
  toJSON() { return '{}'; }
}

```

Static Members

```

class Counter {
  static count = 0;
  static increment(): number { return ++Counter.count; }
}

Counter.increment();

```

```
console.log(Counter.count); // 1
```

Getters and Setters

```
class Temperature {  
  private _celsius = 0;  
  get celsius(): number { return this._celsius; }  
  set celsius(v: number) {  
    if (v < -273.15) throw new Error('Below absolute zero');  
    this._celsius = v;  
  }  
  get fahrenheit(): number { return this._celsius * 9/5 + 32; }  
}  
  
const t = new Temperature();  
t.celsius = 100;  
console.log(t.fahrenheit); // 212
```

Chapter 10: Generics

Generics let you write reusable, type-safe code that works with multiple types without sacrificing type information. They are non-negotiable knowledge at L2.

The Problem Without Generics

```
// Without generics – we lose type info
function identity(arg: any): any { return arg; }
const x = identity('hello'); // x is 'any', not 'string'

// With generics – type flows through
function identity<T>(arg: T): T { return arg; }
const y = identity('hello'); // y is 'string'
const z = identity(42);      // z is 'number'
```

Generic Functions

```
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}

first([1,2,3]);           // T = number
first(['a','b','c']);     // T = string

// Multiple type parameters
function pair<A, B>(a: A, b: B): [A, B] { return [a, b]; }
pair('Ada', 30);          // [string, number]
```

Generic Constraints (extends)

```
// Require T to have a 'length' property
function logLength<T extends { length: number }>(arg: T): T {
  console.log(arg.length);
  return arg;
}

logLength('hello');      // OK – string has length
logLength([1, 2, 3]);    // OK
// logLength(42);        // Error – number has no length
```

Default Type Parameters

```
interface ApiResponse<T = unknown> {
  status: number;
  data: T;
}

const r1: ApiResponse = { status: 200, data: 'anything' };
const r2: ApiResponse<{ id: number }> = { status: 200, data: { id: 1 } };
```

Generic Interfaces and Classes

```
// Generic interface
interface Repository<T> {
  findById(id: number): T | null;
  save(entity: T): void;
  findAll(): T[];
}

// Generic class
class Stack<T> {
  private items: T[] = [];
}
```

```

    push(item: T): void { this.items.push(item); }
    pop(): T | undefined { return this.items.pop(); }
    peek(): T | undefined { return this.items[this.items.length - 1]; }
    get size(): number { return this.items.length; }
}

```

```

const numbers = new Stack<number>();
numbers.push(1);
numbers.push(2);
const top = numbers.pop(); // number | undefined

```

keyof with Generics — A Powerful Idiom

```

function getProp<T, K extends keyof T>(obj: T, key: K): T[K] {
    return obj[key];
}

const user = { name: 'Ada', age: 30 };
const name = getProp(user, 'name'); // type: string
const age = getProp(user, 'age');    // type: number
// getProp(user, 'email');           // Error – 'email' not in keyof user

```

Generic Type Constraints in Practice

```

// Merge two objects
function merge<T extends object, U extends object>(a: T, b: U): T & U {
    return { ...a, ...b };
}

const merged = merge({ name: 'Ada' }, { age: 30 });
// merged: { name: string } & { age: number }
console.log(merged.name, merged.age);

```

Chapter 11: Utility Types

TypeScript ships built-in utility types that transform existing types. Knowing these cold separates juniors from associate engineers.

Partial<T> — Make All Properties Optional

```
interface User { id: number; name: string; email: string }

function update(id: number, changes: Partial<User>) { /* ... */ }
update(1, { name: 'New Name' }); // only some fields
```

Required<T> — Make All Properties Required

```
interface Config { host?: string; port?: number }
type FullConfig = Required<Config>;
// FullConfig = { host: string; port: number }
```

Readonly<T> — Make All Properties readonly

```
const frozen: Readonly<User> = { id: 1, name: 'Ada', email: 'a@b.com' };
// frozen.name = 'X'; // Error
```

Pick<T, K> — Select Specific Properties

```
type UserPreview = Pick<User, 'id' | 'name'>;
// UserPreview = { id: number; name: string }
```

Omit<T, K> — Exclude Specific Properties

```
type UserCreate = Omit<User, 'id'>;
// UserCreate = { name: string; email: string }
```

Record<K, V> — Build Object Type with Specific Keys

```
type Role = 'admin' | 'editor' | 'viewer';
type Permissions = Record<Role, string[]>;
// Permissions = { admin: string[]; editor: string[]; viewer: string[] }

const perms: Permissions = {
  admin: ['read', 'write', 'delete'],
  editor: ['read', 'write'],
  viewer: ['read']
};
```

Exclude<T, U> and Extract<T, U>

```
type T1 = Exclude<'a' | 'b' | 'c', 'a'>; // 'b' | 'c'
type T2 = Extract<'a' | 'b' | 'c', 'a' | 'd'>; // 'a'
type T3 = Exclude<string | number | (() => void), Function>; // string | number
```

NonNullable<T>

```
type T = NonNullable<string | null | undefined>; // string
```

Function-Related Utilities

```
function getUser(id: number, opts?: { full?: boolean }) {
  return { id, name: 'Ada' };
}

type Params = Parameters<typeof getUser>;
// [number, { full?: boolean }?]
```



```

type Return = ReturnType<typeof getUser>;
// { id: number; name: string }

// For methods
class A { m(x: number): string { return ''; } }
type MParams = Parameters<A['m']>;           // [number]
type MReturn = ReturnType<A['m']>;           // string

// Promises
async function fetchUser() { return { id: 1, name: 'Ada' }; }
type UserData = Awaited<ReturnType<typeof fetchUser>>;
// { id: number; name: string }

```

String Manipulation Utilities (TS 4.1+)

```

type T1 = Uppercase<'hello'>;    // 'HELLO'
type T2 = Lowercase<'HELLO'>;   // 'hello'
type T3 = Capitalize<'hello'>;  // 'Hello'
type T4 = Uncapitalize<'Hello'>; // 'hello'

// Template literal types
type Event = 'click' | 'hover';
type Handler = `on${Capitalize<Event>}`;
// 'onClick' | 'onHover'

```

Chapter 12: Modules, Namespaces, and tsconfig

ES Modules

```
// math.ts
export function add(a: number, b: number): number { return a + b; }
export function sub(a: number, b: number): number { return a - b; }
export const PI = 3.14159;

// Default export
export default function square(x: number): number { return x * x; }

// index.ts
import square, { add, sub, PI } from './math';
import * as M from './math'; // namespace import
import { add as plus } from './math'; // rename
```

Re-exports

```
// barrel file – index.ts
export { User } from './User';
export { Post } from './Post';
export * from './common';
```

type-only Imports/Exports (TS 3.8+)

```
// Pure type imports are erased – useful for build optimization
import type { User } from './User';
import { type Config, makeConfig } from './config';
```

Namespaces (Legacy)

Namespaces predate ES modules. Prefer modules in modern code. You'll see namespaces mostly in older codebases and ambient declaration files.

```
namespace Validation {
  export interface StringValidator { isAcceptable(s: string): boolean }
  export class LettersOnly implements StringValidator {
    isAcceptable(s: string) { return /^[A-Za-z]+$/.test(s); }
  }
}

const v = new Validation.LettersOnly();
```

Declaration Files (.d.ts)

.d.ts files contain only type declarations — no implementation. They describe the shape of JavaScript libraries to TypeScript. The DefinitelyTyped repository (**@types/*** packages on npm) contains community-maintained .d.ts files for thousands of JS libraries.

```
// install types for a JS library
npm install --save-dev @types/lodash

// custom global declarations – globals.d.ts
declare const APP_VERSION: string;
declare module '*.svg' {
  const url: string;
  export default url;
}
```

Module Resolution Strategies

- **"node" / "node10"** — classic Node resolution.
- **"node16" / "nodenext"** — modern Node with ESM support.
- **"bundler"** — for bundlers (Vite, esbuild, webpack). TypeScript 5.0+.

Path Aliases

```
// tsconfig.json
{
  "compilerOptions": {
    "baseUrl": "./src",
    "paths": {
      "@/*": ["*"],
      "@components/*": ["components/*"],
      "@utils/*": ["utils/*"]
    }
  }
}

// Now you can write:
// import { Button } from '@components/Button';
// instead of
// import { Button } from '../../components/Button';
```

Common tsconfig Options Reference

- **target** — ECMAScript version of the emitted JS (ES2022 is a safe modern pick).
- **module** — module system: commonjs (Node), esnext (modern), nodenext.
- **strict** — master switch for all strict checks.
- **noEmit** — type-check only, don't write JS (used with bundlers).
- **jsx** — react / react-jsx / preserve (for React projects).
- **allowJs / checkJs** — incrementally adopt TS in JS codebases.
- **incremental** — speeds up rebuilds with .tsbuildinfo cache.
- **isolatedModules** — required when using bundlers like Babel/SWC.

Part III — Advanced TypeScript

Chapter 13: The Advanced Type System

typeof Operator (in type position)

Used to get the type of a value/variable. Different from JavaScript's runtime typeof.

```
const config = {
  host: 'localhost',
  port: 3000,
  ssl: false
};

type Config = typeof config;
// type Config = { host: string; port: number; ssl: boolean }

function f(): { a: number } { return { a: 1 }; }
type R = ReturnType<typeof f>;
```

keyof Operator

```
interface User { id: number; name: string; email: string }
type UserKey = keyof User;
// type UserKey = 'id' | 'name' | 'email'

// keyof + typeof combo – very common
const themes = { light: '#fff', dark: '#000', sepia: '#f4ecd8' };
type ThemeName = keyof typeof themes; // 'light' | 'dark' | 'sepia'
```

Indexed Access Types

```
interface User { id: number; name: string; address: { city: string } }

type UserId = User['id']; // number
type UserCity = User['address']['city']; // string

// Lookup multiple keys (union)
type IdOrName = User['id' | 'name']; // number | string

// All values type
type UserValue = User[keyof User]; // number | string | { city: string }

// Array element type
const arr = ['a', 'b', 'c'];
type Item = typeof arr[number]; // string
```

Mapped Types

Build new types by transforming existing ones. The internal mechanic behind nearly every utility type.

```
// Make all properties optional (this is how Partial is implemented)
type MyPartial<T> = { [K in keyof T]?: T[K] };

// Make all properties readonly
type MyReadOnly<T> = { readonly [K in keyof T]: T[K] };

// Remove optional/readonly modifiers
type MyRequired<T> = { [K in keyof T]-?: T[K] };
type Mutable<T> = { -readonly [K in keyof T]: T[K] };

// Transform value types
```

```

type Nullable<T> = { [K in keyof T]: T[K] | null };

// Key remapping (TS 4.1+) using 'as'
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K]
};
interface Person { name: string; age: number }
type PG = Getters<Person>;
// { getName(): string; getAge(): number }

```

Conditional Types

```

// Basic syntax: T extends U ? X : Y
type IsString<T> = T extends string ? true : false;
type A = IsString<'hi'>; // true
type B = IsString<42>; // false

// Distributive conditional – applies to each union member
type ToArray<T> = T extends any ? T[] : never;
type X = ToArray<string | number>; // string[] | number[]

// infer keyword – extracts a type from a position
type ElementOf<T> = T extends (infer E)[] ? E : never;
type N = ElementOf<number[]>; // number

// Recreating ReturnType
type MyReturnType<T> =
  T extends (...args: any[]) => infer R ? R : never;

```

Template Literal Types

```

type Greeting = `Hello, ${string}!`;
const g1: Greeting = 'Hello, world!'; // OK
// const g2: Greeting = 'Hi'; // Error

// Combinatorial power
type Variant = 'primary' | 'secondary';
type Size = 'small' | 'large';
type ButtonClass = `btn-${Variant}-${Size}`;
// 'btn-primary-small' | 'btn-primary-large' |
// 'btn-secondary-small' | 'btn-secondary-large'

// Type-safe path building
type Method = 'GET' | 'POST' | 'PUT' | 'DELETE';
type Endpoint = '/users' | '/posts';
type Route = `${Method} ${Endpoint}`;

```

Deep Readonly — Combining Concepts

```

type DeepReadonly<T> = {
  readonly [K in keyof T]:
    T[K] extends object ? DeepReadonly<T[K]> : T[K]
};

interface Settings {
  ui: { theme: string; lang: string };
  user: { id: number };
}

const s: DeepReadonly<Settings> = {
  ui: { theme: 'dark', lang: 'en' },
  user: { id: 1 }
};
// s.ui.theme = 'light'; // Error – readonly all the way down

```

Chapter 14: Decorators and Metadata

Decorators are special declarations attached to classes, methods, properties, or parameters. They are used heavily in frameworks like NestJS, Angular, and TypeORM. TypeScript 5.0 introduced the new **Stage 3 decorators**; older code may use the **experimentalDecorators** flag.

Enabling Decorators

```
// tsconfig.json – for legacy / framework-style decorators
{
  "compilerOptions": {
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  }
}
```

Class Decorator (legacy syntax)

```
function sealed(target: Function) {
  Object.seal(target);
  Object.seal(target.prototype);
}

@sealed
class Greeter {
  greet() { return 'hello'; }
}
```

Method Decorator

```
function log(_t: any, name: string, desc: PropertyDescriptor) {
  const original = desc.value;
  desc.value = function (...args: any[]) {
    console.log(`Calling ${name} with`, args);
    return original.apply(this, args);
  };
}

class Calc {
  @log
  add(a: number, b: number): number { return a + b; }
}

new Calc().add(2, 3); // Logs: Calling add with [2, 3]
```

Property Decorator

```
function required(target: any, key: string) {
  let value: any;
  Object.defineProperty(target, key, {
    get() { return value; },
    set(v: any) {
      if (v === undefined || v === null) {
        throw new Error(`${key} is required`);
      }
      value = v;
    }
  });
}

class User {
  @required name!: string;
}
```

Decorator Factory Pattern

```
function route(path: string) {
  return function (target: any, method: string) {
    // Register route in a routing table
    console.log(`Registered ${path} -> ${method}`);
  };
}

class UserController {
  @route('/users')
  list() { /* ... */ }

  @route('/users/:id')
  show() { /* ... */ }
}
```

Note: Frameworks like NestJS make heavy use of decorators for routing (@Get, @Post), dependency injection (@Injectable), validation, and more. Familiarity with decorators is a strong signal at L2 backend interviews.

Chapter 15: Async/Await and Promise Typing

Typing Promises

```
// A function that returns a Promise of T
function fetchUser(id: number): Promise<User> {
    return fetch(`/api/users/${id}`).then(r => r.json());
}

// async functions always return Promise<T>
async function getUser(id: number): Promise<User> {
    const r = await fetch(`/api/users/${id}`);
    return r.json();
}

// The return type can be inferred
async function getUsername(id: number) {
    const u = await getUser(id);
    return u.name;
}
// inferred type: Promise<string>
```

Error Handling

```
async function safeFetch(url: string): Promise<unknown> {
    try {
        const r = await fetch(url);
        if (!r.ok) throw new Error(`HTTP ${r.status}`);
        return await r.json();
    } catch (err) {
        // In strict mode, err is typed as 'unknown' (TS 4.4+)
        if (err instanceof Error) {
            console.error(err.message);
        }
        throw err;
    }
}
```

Promise.all, Promise.allSettled, Promise.race

```
// Promise.all – fail-fast, returns a tuple of results
const [user, posts] = await Promise.all([
    fetchUser(1),
    fetchPosts(1)
]); // type: [User, Post[]]

// Promise.allSettled – never throws, returns status for each
const results = await Promise.allSettled([fetchUser(1), fetchUser(2)]);
results.forEach(r => {
    if (r.status === 'fulfilled') console.log(r.value);
    else console.error(r.reason);
});

// Promise.race – first to settle wins
const fastest = await Promise.race([fetchUser(1), fetchUser(2)]);
```

Awaited<T> — Recursive Unwrap

```
// Unwraps nested promises
type A = Awaited<Promise<string>>;           // string
type B = Awaited<Promise<Promise<number>>>; // number
```

Typing fetch Responses Safely


```
interface User { id: number; name: string }

async function getUser(id: number): Promise<User> {
  const r = await fetch(`/api/users/${id}`);
  if (!r.ok) throw new Error('Failed');
  const data: unknown = await r.json();

  // Don't trust 'json()' – validate at the boundary!
  if (
    typeof data === 'object' && data !== null &&
    'id' in data && 'name' in data
  ) {
    return data as User;
  }
  throw new Error('Invalid user payload');
}
```

Note: In production, use a validation library like **Zod** or **io-ts** instead of hand-rolled guards. They give you both runtime validation AND inferred TypeScript types.

Part IV — Full Stack TypeScript

Chapter 16: TypeScript with Node.js and Express

Project Setup

```
mkdir api && cd api
npm init -y
npm install express
npm install --save-dev typescript @types/node @types/express ts-node-dev
npx tsc --init
```

Minimal Express Server

```
// src/index.ts
import express, { Request, Response, NextFunction } from 'express';

const app = express();
app.use(express.json());

app.get('/health', (req: Request, res: Response) => {
  res.json({ status: 'ok' });
});

app.listen(3000, () => console.log('listening on 3000'));
```

Typed Request/Response with Generics

```
// Express generics: Request<Params, ResBody, ReqBody, Query>
interface CreateUserBody { name: string; email: string }
interface UserResponse { id: number; name: string; email: string }
interface PathParams { id: string }

app.post(
  '/users',
  (req: Request<{}, UserResponse, CreateUserBody>, res: Response<UserResponse>) => {
    const { name, email } = req.body; // fully typed
    const user = { id: 1, name, email };
    res.status(201).json(user);
  }
);

app.get('/users/:id', (req: Request<PathParams>, res: Response) => {
  const id = Number(req.params.id); // params are always string
  res.json({ id, name: 'Ada' });
});
```

Error-Handling Middleware

```
interface AppError extends Error {
  status?: number;
  code?: string;
}

function errorHandler(
  err: AppError,
  _req: Request,
  res: Response,
  _next: NextFunction
) {
  res.status(err.status ?? 500).json({
    error: err.message,
  });
}
```

```

        code: err.code ?? 'INTERNAL'
    });
}

app.use(errorHandler);

```

Async Route Handlers

```

// Wrap async handlers so thrown errors reach error middleware
type Handler = (req: Request, res: Response, next: NextFunction) => Promise<void>;

const asyncHandler = (fn: Handler) =>
  (req: Request, res: Response, next: NextFunction) => {
    Promise.resolve(fn(req, res, next)).catch(next);
  };

app.get('/users/:id', asyncHandler(async (req, res) => {
  const user = await db.users.findById(Number(req.params.id));
  if (!user) throw Object.assign(new Error('Not found'), { status: 404 });
  res.json(user);
})));

```

Environment Variables (Type-Safe)

```

// src/env.ts – validate & type your env vars
import { z } from 'zod';

const schema = z.object({
  NODE_ENV: z.enum(['development', 'test', 'production']),
  PORT: z.coerce.number().default(3000),
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32)
});

export const env = schema.parse(process.env);
// Now env.PORT is number, env.DATABASE_URL is string, etc.

```

Layered Project Structure

```

src/
  controllers/  # request handlers
  services/    # business logic
  repositories/ # data access
  models/      # type definitions
  middleware/  # auth, logging, errors
  routes/      # route definitions
  utils/       # helpers
  index.ts     # entry point

```

Chapter 17: TypeScript with React

Setup

```
# Vite – fastest path
npm create vite@latest my-app -- --template react-ts

# Or Next.js
npx create-next-app@latest --typescript
```

Typing Props

```
// Define props as an interface
interface ButtonProps {
  label: string;
  variant?: 'primary' | 'secondary';
  disabled?: boolean;
  onClick: () => void;
}

// Function component
function Button({ label, variant = 'primary', disabled, onClick }: ButtonProps) {
  return (
    <button className={`btn btn-${variant}`} disabled={disabled} onClick={onClick}>
      {label}
    </button>
  );
}
```

Children

```
import { ReactNode, PropsWithChildren } from 'react';

// Explicit
interface CardProps { title: string; children: ReactNode }
function Card({ title, children }: CardProps) {
  return <div><h3>{title}</h3>{children}</div>;
}

// With helper
function Section({ title, children }: PropsWithChildren<{ title: string }>) {
  return <section><h2>{title}</h2>{children}</section>;
}
```

useState

```
import { useState } from 'react';

// Primitive – inferred
const [count, setCount] = useState(0); // number

// Object – usually inferred from initial value
const [user, setUser] = useState({ name: '', age: 0 });

// Nullable – provide the type explicitly
const [user2, setUser2] = useState<User | null>(null);

// Array of objects
const [items, setItems] = useState<Item[]>([]);
```

Event Handlers

```
import { ChangeEvent, FormEvent, MouseEvent } from 'react';
```

```

function Form() {
  const [email, setEmail] = useState('');

  const handleChange = (e: ChangeEvent<HTMLInputElement>) => {
    setEmail(e.target.value);
  };

  const handleSubmit = (e: FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    console.log(email);
  };

  const handleClick = (e: MouseEvent<HTMLButtonElement>) => {
    console.log(e.currentTarget.tagName);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={handleChange} />
      <button onClick={handleClick}>Send</button>
    </form>
  );
}

```

useEffect, useRef, useReducer

```

import { useEffect, useRef, useReducer } from 'react';

// useEffect – no special types needed
useEffect(() => {
  const id = setInterval(() => console.log('tick'), 1000);
  return () => clearInterval(id);
}, []);

// useRef on DOM
const inputRef = useRef<HTMLInputElement>(null);
useEffect(() => { inputRef.current?.focus(); }, []);

// useReducer with discriminated union
type State = { count: number };
type Action = { type: 'inc' } | { type: 'dec' } | { type: 'reset' };

function reducer(s: State, a: Action): State {
  switch (a.type) {
    case 'inc': return { count: s.count + 1 };
    case 'dec': return { count: s.count - 1 };
    case 'reset': return { count: 0 };
  }
}

const [state, dispatch] = useReducer(reducer, { count: 0 });

```

Custom Hooks

```

function useFetch<T>(url: string): {
  data: T | null;
  loading: boolean;
  error: Error | null;
} {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<Error | null>(null);

  useEffect(() => {
    fetch(url)
      .then(r => r.json())

```

```

        .then(setData)
        .catch(setError)
        .finally(() => setLoading(false));
    }, [url]);

    return { data, loading, error };
}

// Usage
const { data, loading } = useFetch<User[]>('/api/users');

```

Context API

```

import { createContext, useContext, ReactNode } from 'react';

interface ThemeContextValue {
    theme: 'light' | 'dark';
    toggle: () => void;
}

const ThemeContext = createContext<ThemeContextValue | null>(null);

export function ThemeProvider({ children }: { children: ReactNode }) {
    const [theme, setTheme] = useState<'light' | 'dark'>('light');
    const toggle = () => setTheme(t => t === 'light' ? 'dark' : 'light');
    return (
        <ThemeContext.Provider value={{ theme, toggle }}>
            {children}
        </ThemeContext.Provider>
    );
}

export function useTheme() {
    const ctx = useContext(ThemeContext);
    if (!ctx) throw new Error('useTheme must be used inside ThemeProvider');
    return ctx;
}

```

Chapter 18: TypeScript with Databases

Most modern Node.js codebases reach for an ORM or query builder with TypeScript support. Prisma is the most popular today; TypeORM is common in NestJS shops; Drizzle is the fastest-growing newcomer.

Prisma — Schema-First

```
// prisma/schema.prisma
datasource db { provider = "postgresql" url = env("DATABASE_URL") }
generator client { provider = "prisma-client-js" }

model User {
  id      Int      @id @default(autoincrement())
  email   String   @unique
  name    String
  posts   Post[]
  createdAt DateTime @default(now())
}

model Post {
  id      Int      @id @default(autoincrement())
  title   String
  content String?
  authorId Int
  author  User     @relation(fields: [authorId], references: [id])
}
```

After running **`npx prisma generate`**, you get a fully typed client:

```
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();

// Create – input is type-checked against the model
const u = await prisma.user.create({
  data: { email: 'a@b.com', name: 'Ada' }
});

// Query with relations
const userWithPosts = await prisma.user.findUnique({
  where: { id: 1 },
  include: { posts: true }
});

// Update
await prisma.user.update({
  where: { id: 1 },
  data: { name: 'Ada Lovelace' }
});

// Delete
await prisma.user.delete({ where: { id: 1 } });
```

TypeORM — Decorator-Based

```
import 'reflect-metadata';
import { Entity, PrimaryGeneratedColumn, Column, OneToMany } from 'typeorm';

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id!: number;

  @OneToMany()
  posts: Post[];
}
```

```

@Column({ unique: true })
email!: string;

@Column()
name!: string;

@OneToMany(() => Post, post => post.author)
posts!: Post[];
}

```

Repository Pattern

```

interface Repository<T, ID> {
  findById(id: ID): Promise<T | null>;
  findAll(): Promise<T[]>;
  create(data: Omit<T, 'id'>): Promise<T>;
  update(id: ID, data: Partial<T>): Promise<T>;
  delete(id: ID): Promise<void>;
}

class UserRepository implements Repository<User, number> {
  constructor(private prisma: PrismaClient) {}
  findById(id: number) { return this.prisma.user.findUnique({ where: { id } }); }
  findAll() { return this.prisma.user.findMany(); }
  create(data: Omit<User, 'id'>) { return this.prisma.user.create({ data }); }
  update(id: number, data: Partial<User>) { return this.prisma.user.update({ where: { id }, data }); }
  async delete(id: number) { await this.prisma.user.delete({ where: { id } }); }
}

```

DTOs (Data Transfer Objects)

```

// Domain entity – what's in the DB
interface User {
  id: number; email: string; name: string;
  passwordHash: string; createdAt: Date;
}

// What clients send
type CreateUserDTO = Pick<User, 'email' | 'name'> & { password: string };

// What clients receive (no password hash!)
type UserDTO = Omit<User, 'passwordHash'>;

function toDTO(u: User): UserDTO {
  const { passwordHash, ...dto } = u;
  return dto;
}

```


Chapter 19: Error Handling, Testing, and Best Practices

Custom Error Classes

```
class AppError extends Error {
  constructor(
    message: string,
    public statusCode: number = 500,
    public code: string = 'INTERNAL_ERROR'
  ) {
    super(message);
    this.name = this.constructor.name;
    Error.captureStackTrace(this, this.constructor);
  }
}

class NotFoundError extends AppError {
  constructor(resource: string) {
    super(`${resource} not found`, 404, 'NOT_FOUND');
  }
}

class ValidationError extends AppError {
  constructor(public fields: Record<string, string>) {
    super('Validation failed', 400, 'VALIDATION_ERROR');
  }
}
```

Result Type Pattern (Functional Error Handling)

```
type Result<T, E = Error> =
  | { ok: true; value: T }
  | { ok: false; error: E };

function divide(a: number, b: number): Result<number, string> {
  if (b === 0) return { ok: false, error: 'Division by zero' };
  return { ok: true, value: a / b };
}

const r = divide(10, 0);
if (r.ok) console.log(r.value);
else console.error(r.error);
```

Testing with Jest

```
npm install --save-dev jest ts-jest @types/jest
npx ts-jest config:init

// math.ts
export function add(a: number, b: number): number { return a + b; }

// math.test.ts
import { add } from './math';

describe('add', () => {
  it('adds two positive numbers', () => {
    expect(add(2, 3)).toBe(5);
  });
  it('adds with negatives', () => {
    expect(add(-2, 3)).toBe(1);
  });
});
```

Mocking with Type Safety

```
// Using jest.fn() with types
type Fetcher = (url: string) => Promise<{ data: unknown }>;

const fetcher: jest.MockedFunction<Fetcher> = jest.fn();
fetcher.mockResolvedValue({ data: { id: 1, name: 'Ada' } });

// Mocking a module
jest.mock('./api');
import * as api from './api';
(api.getUser as jest.Mock).mockResolvedValue({ id: 1 });
```

Best Practices Checklist

- Enable **strict: true** in tsconfig — non-negotiable for new projects.
- Avoid **any**. Use **unknown** for boundary code and narrow it.
- Lean on type inference; don't annotate what the compiler can infer.
- Use **readonly** for arrays/properties that shouldn't mutate.
- Validate external data (API responses, env vars) at runtime with Zod or similar.
- Prefer discriminated unions over inheritance for state modeling.
- Use **const assertions** for literal-precision: **as const**.
- Group related types in dedicated files — keep modules focused.
- Co-locate component types with components (React) and DTOs with services.
- Run **tsc --noEmit** in CI to catch type errors that linters miss.
- Adopt ESLint + **@typescript-eslint** for stylistic + correctness rules.
- Use Prettier — never argue about formatting again.

Part V — Interview Preparation

Chapter 20: L1 Interview Questions (Entry-Level)

L1 questions focus on fundamentals: do you understand the language? Can you read TypeScript code and explain it? Most L1 interviews mix conceptual Q&A with one or two small coding exercises.

Q1. What is TypeScript and how is it different from JavaScript?

TypeScript is a strongly-typed superset of JavaScript developed by Microsoft. It adds optional static types, interfaces, enums, and modern language features. TypeScript code compiles down to JavaScript. The key advantage is catching bugs at compile time rather than runtime, plus richer IDE support and self-documenting code.

Q2. What does "superset of JavaScript" mean?

Every valid JavaScript program is also a valid TypeScript program. You can rename any .js file to .ts and it will compile (though with warnings on missing types). TypeScript adds features on top of JS without taking any away.

Q3. What is the difference between any and unknown?

Both can hold any value. The difference: any disables type checking — you can access any property or call any method without errors. unknown is type-safe — you must narrow it (typeof, instanceof, or a custom type guard) before using it. Use unknown for incoming data of unknown shape; reserve any only for migration.

Q4. What is the never type?

never represents a value that can never occur. It is the return type of functions that throw or loop forever, and the type of variables narrowed away by control flow. It's also useful for exhaustiveness checking in switch statements.

Q5. void vs undefined — what's the difference?

void signals that a function returns no meaningful value — but the function may still return undefined (or nothing). undefined as a return type means the function must explicitly return undefined. In callback positions void is more permissive.

Q6. What are type aliases and how do they differ from interfaces?

Both describe object shapes. Type aliases use the 'type' keyword and can describe anything: unions, intersections, primitives, tuples, mapped types. Interfaces use the 'interface' keyword and are limited to object shapes — but they support declaration merging (re-opening). Prefer interface for object shapes; use type for unions and computed types.

Q7. What is type inference?

TypeScript can deduce types from context without explicit annotations. For example, let x = 5 infers x as number. Inference covers variables, return types, generic parameters, and contextual types (like callback parameter types). Idiomatic TypeScript leans heavily on inference and adds annotations only where helpful.

Q8. What is the difference between a tuple and an array?

An array is a collection of elements of the same type with variable length. A tuple is a fixed-length collection where each position has its own type, e.g. [string, number]. Tuples are great for returning multiple typed values from a function (think React's useState).

Q9. What are union and intersection types?

Union (|) means 'one of'. string | number means either a string or a number. Intersection (&) means 'all of'. A & B has all properties of A AND B. Unions are more common in everyday code; intersections shine when composing types.

Q10. What are optional and default parameters?

Optional parameters use ? after the name and may be undefined. Default parameters provide a value if the argument is omitted, making the parameter implicitly optional. Both must come after required parameters.

Q11. What is type assertion?

Type assertion tells the compiler to treat a value as a specific type, using either **value as Type** or angle-bracket **<Type>value** syntax (not allowed in .tsx). It does NOT perform runtime conversion — it only changes how the compiler views the value. Use it sparingly; if you're wrong, you get runtime errors.

Q12. What is a readonly property?

A property declared readonly can only be assigned during initialization (constructor or property declaration). It cannot be reassigned afterwards. Note: this is a compile-time check only — the underlying JS object is still mutable at runtime.

Q13. What is an enum? When would you use one?

An enum defines a set of named constants. Numeric enums auto-increment from 0; string enums require explicit values. Useful for representing a fixed set of choices like status codes or directions. Modern alternatives include union-of-literal types and 'as const' objects, which many teams prefer because they don't emit runtime code.

Q14. What is the difference between == and === in TypeScript?

Identical to JavaScript: == performs type coercion, === does not. TypeScript's strict mode and most lint configurations enforce ===. Prefer === unless you have an explicit reason to compare loosely.

Q15. How does TypeScript handle null and undefined?

With strictNullChecks enabled, null and undefined are distinct types and are NOT assignable to other types unless explicitly included. You must either narrow or use the non-null assertion (!) when you're sure a value isn't null/undefined. Modern TypeScript always uses strictNullChecks.

Q16. What does the ! non-null assertion operator do?

It tells the compiler that a value is not null or undefined — without performing a runtime check. Useful when the compiler can't see what you can. Use sparingly; you're disabling a safety check.

Q17. What is the spread operator and how is it typed?

The spread operator (...) expands arrays/objects. In TypeScript, spreading is type-safe — spread an array of T into a function expecting (T, T, T) for example. Object spread merges types via intersection.

Q18. Explain function overloading in TypeScript.

You declare multiple overload signatures followed by a single implementation. Callers see only the overloads. Useful when a function accepts genuinely different argument shapes that need different return types. Frequently overused — try a union type or generics first.

Q19. What is a type guard?

A type guard is an expression that narrows a value's type within a scope. Built-in guards include typeof, instanceof, in, and equality checks. You can write user-defined guards: a function returning **value is Type** narrows on success.

Q20. What is the spread/rest operator distinction?

They use the same ... syntax. **Rest** collects: function f(...args: number[]) gathers arguments into an array. **Spread** expands: f(...numbers) passes elements as individual arguments. Context determines which it is.

Chapter 21: L2 Interview Questions (Associate Engineer)

L2 interviews probe deeper: generics, advanced types, design choices, and real-world scenarios. Expect to explain TRADE-OFFS, not just facts. Below are 25 commonly-asked questions with model answers.

Q1. What are generics? Give a real-world example.

Generics let you write reusable, type-safe code that works with multiple types without losing type information. Example: a generic `Stack<T>` class can store numbers, strings, or `User` objects, with type-safe push/pop operations for each instance. Without generics, you'd either duplicate code or fall back to `any`.

Q2. Explain extends in generic constraints.

The `extends` keyword restricts a generic type parameter. `T extends { length: number }` means `T` must have a `length` property — strings and arrays qualify, numbers don't. Constraints unlock property access inside the generic function while preserving the input's specific type.

Q3. Walk through the implementation of Partial<T>.

`Partial` is a mapped type: `type Partial<T> = { [K in keyof T]?: T[K] }`; It iterates over each key of `T` using `keyof`, and makes that property optional with `?`. It's the simplest mapped type and the gateway to understanding the rest.

Q4. What is the difference between Pick and Omit?

`Pick` selects specific properties to keep. `Omit` excludes properties. `Pick<User, 'id' | 'name'>` gives you a type with only `id` and `name`; `Omit<User, 'password'>` gives you everything EXCEPT `password`. Both take a type and a union of keys.

Q5. How would you implement DeepPartial?

A recursive mapped type: `type DeepPartial<T> = { [K in keyof T]?: T[K] extends object ? DeepPartial<T[K]> : T[K] }`; It makes every property optional, and if the value is an object, recursively applies `DeepPartial`.

Q6. What is a discriminated union? Why use one?

A union where each member has a common literal property (the discriminant). TypeScript can narrow within the union based on this property's value. It models state machines and variant data cleanly — for example, `{ kind: 'success', data } | { kind: 'error', message }`. Pairs perfectly with exhaustive switch checking using `never`.

Q7. What is exhaustiveness checking? Show how never enables it.

Inside a switch on a discriminated union, the default branch can assign the narrowed value to a `never` typed variable. If a new variant is added without updating the switch, the value is no longer `never` and the compiler errors. This catches missing cases at compile time.

Q8. Explain conditional types.

A conditional type chooses one of two types based on a condition: `T extends U ? X : Y`. They distribute over unions by default (each member is checked independently). With `infer`, you can

extract types from positions, which is how `ReturnType`, `Parameters`, and `Awaited` are implemented.

Q9. What does the `infer` keyword do?

`infer` introduces a type variable in a conditional type that captures a type at a specific position. **`type ElementOf<T> = T extends (infer E)[] ? E : never`**; captures the element type of an array. `infer` is the foundation of advanced type inference and powers `Awaited`, `ReturnType`, `Parameters`, and many others.

Q10. How does structural typing work in TypeScript?

TypeScript uses structural (a.k.a. duck) typing. Two types are compatible if their members are compatible — regardless of declared names. A function expecting a type with `id` and `name` accepts any object that has `id` and `name`, even if it's typed differently. This contrasts with nominal typing (Java/C#) where names matter.

Q11. What is declaration merging?

Two declarations with the same name (e.g. two interfaces, or an interface + a namespace) are merged by the compiler into a single declaration. Used to extend third-party types (like adding a property to Express's `Request`) and module augmentation. Only interfaces (not type aliases) can be merged.

Q12. What is module augmentation? Give an example.

Adding declarations to an existing module without modifying its source. Common case: extending Express's `Request` with a custom user property:

```
// global.d.ts
import { User } from './models/User';
declare global {
  namespace Express {
    interface Request {
      user?: User;
    }
  }
}
export {};
```

// Now req.user is typed throughout the codebase.

Q13. Compare type aliases and interfaces for OOP design.

Interfaces signal a contract you implement (good for classes via implements). They support declaration merging — useful when extending library types. Type aliases are more flexible and required for unions, tuples, and computed types. Practical rule: interface for public-facing object shapes and class contracts; type for everything else.

Q14. What is covariance and contravariance in function types?

Function parameter types are checked contravariantly (the parameter of the supertype must be assignable to the parameter of the subtype) and return types covariantly. TypeScript's `--strictFunctionTypes` enforces this correctly. Most engineers don't need this day-to-day, but it explains some surprising type errors.

Q15. What's the difference between an abstract class and an interface?

An abstract class can contain implementation (concrete methods, constructors, state) alongside abstract methods. An interface contains only declarations. A class can implement many interfaces but extend only one (abstract) class. Use abstract classes when you want shared behavior; use interfaces when you want a pure contract.

Q16. What are utility types? Name 6 and what they do.

Built-in type transformations: Partial (all optional), Required (all required), Readonly (all readonly), Pick (select keys), Omit (exclude keys), Record (build from key+value types), Exclude/Extract (filter unions), NonNullable (drop null/undefined), ReturnType, Parameters, Awaited.

Q17. What is keyof typeof obj? Why is it useful?

typeof obj gives the type of a runtime value. **keyof X** gives a union of X's property names. Combined, **keyof typeof obj** gives the union of an object's keys as a string-literal type. Common for constraining function arguments to valid keys of an object, or extracting enum-like types from 'as const' objects.

Q18. How would you create a type-safe event emitter?

Use a generic interface that maps event names (keys) to payload types (values), then constrain emit/on with keyof:

```
type EventMap = {
  login: { userId: number };
  logout: { userId: number };
  error: { message: string };
};

class TypedEmitter<T extends Record<string, any>> {
  private handlers: { [K in keyof T]?: ((p: T[K]) => void)[] } = {};

  on<K extends keyof T>(event: K, fn: (p: T[K]) => void) {
    (this.handlers[event] ??= []).push(fn);
  }
  emit<K extends keyof T>(event: K, payload: T[K]) {
    this.handlers[event]?.forEach(fn => fn(payload));
  }
}

const ee = new TypedEmitter<EventMap>();
ee.on('login', p => console.log(p.userId)); // p is typed
ee.emit('error', { message: 'oops' });      // payload is enforced
```

Q19. How do you safely parse JSON from an API in TypeScript?

Treat the parsed value as unknown, then validate. Either with a hand-written type guard, or with a runtime validation library like Zod that infers a TypeScript type from the schema and validates at runtime in one step.

Q20. What is the satisfies operator (TS 4.9+)?

satisfies checks that a value matches a type WITHOUT widening the value's inferred type. Useful when you want both validation AND precise inference. Example: **const config = { host: 'x', port: 80 } satisfies Config;** — config.port is still the literal type 80, not just number.

Q21. What is the difference between unknown and any when receiving unknown input?

any propagates — once you have any, everything derived from it is also any. unknown is contained — you cannot do anything with it until you narrow it. Use unknown at API boundaries to force validation; never use any unless temporarily during migration.

Q22. How would you type a function that depends on the type of its argument?

Use generics with conditional return types, or function overloads. For instance:

```
function unwrap<T>(v: T | null | undefined): T {
  if (v == null) throw new Error('Empty');
  return v;
}

// Or conditional return type
type UnArray<T> = T extends (infer U)[] ? U : T;
function first<T>(arr: T): UnArray<T> {
  return (Array.isArray(arr) ? arr[0] : arr) as UnArray<T>;
}
```

Q23. How would you design a type-safe API client?

Define a route map (object describing each endpoint's path, method, request, and response types), then write a generic client.call(routeName, params) that infers the request and response types from the route name. Libraries like tRPC and ts-rest take this further with end-to-end type-safety from server to client.

Q24. What are the trade-offs of using TypeScript?

Pros: compile-time safety, better tooling, easier refactoring, self-documenting APIs, scaling teams. Cons: extra build step, learning curve, occasional friction with untyped libraries, longer initial setup, types don't validate runtime data. Net positive for almost any project beyond a small script.

Q25. How does TypeScript improve refactoring?

Renaming a property, removing a function argument, or changing a return type surfaces all affected callers as compile errors immediately. The IDE can perform safe rename-symbol operations across the entire codebase. This is one of the largest practical wins in mid-to-large projects.

Chapter 22: Coding Challenges with Solutions

These are the kinds of small, focused coding tasks you'll see in screenings and live interviews. Read each problem first, attempt it yourself, then check the solution.

Challenge 1: Type-Safe getProperty

Write a generic function that safely gets a property from an object given a key.

```
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const user = { name: 'Ada', age: 30 };
const name = getProperty(user, 'name'); // string
const age = getProperty(user, 'age');    // number
// getProperty(user, 'email');           // Error
```

Challenge 2: Implement Pick<T, K>

```
type MyPick<T, K extends keyof T> = {
  [P in K]: T[P]
};

interface Todo { title: string; description: string; completed: boolean }
type Preview = MyPick<Todo, 'title' | 'completed'>;
// { title: string; completed: boolean }
```

Challenge 3: Tuple to Object

```
type TupleToObject<T extends readonly (string | number)[]> = {
  [K in T[number]]: K
};

const tuple = ['tesla', 'model 3', 'model x'] as const;
type R = TupleToObject<typeof tuple>;
// { tesla: 'tesla'; 'model 3': 'model 3'; 'model x': 'model x' }
```

Challenge 4: First Element of Array Type

```
type First<T extends any[]> = T extends [infer F, ...any[]] ? F : never;

type A = First<[1, 2, 3]>; // 1
type B = First<[]>;       // never
```

Challenge 5: Length of Tuple

```
type Length<T extends readonly any[]> = T['length'];

type N = Length<[1, 2, 3, 4]>; // 4
```

Challenge 6: Implement Awaited

```
type MyAwaited<T> =
  T extends Promise<infer U>
    ? U extends Promise<any> ? MyAwaited<U> : U
    : T;

type X = MyAwaited<Promise<Promise<string>>>; // string
```

Challenge 7: Deep Clone (Typed)

```
function deepClone<T>(value: T): T {
```

```

    if (value === null || typeof value !== 'object') return value;
    if (Array.isArray(value)) {
        return value.map(v => deepClone(v)) as unknown as T;
    }
    const out: Record<string, unknown> = {};
    for (const [k, v] of Object.entries(value as object)) {
        out[k] = deepClone(v);
    }
    return out as T;
}

```

Challenge 8: Debounce — Type the Returned Function

```

function debounce<T extends (...args: any[]) => any>(
    fn: T,
    wait: number
): (...args: Parameters<T>) => void {
    let timer: ReturnType<typeof setTimeout> | undefined;
    return (...args: Parameters<T>) => {
        clearTimeout(timer);
        timer = setTimeout(() => fn(...args), wait);
    };
}

const log = (n: number, s: string) => console.log(n, s);
const debouncedLog = debounce(log, 300);
debouncedLog(1, 'hi'); // arguments fully typed

```

Challenge 9: Linked List Class (Generic)

```

class Node<T> {
    constructor(public value: T, public next: Node<T> | null = null) {}
}

class LinkedList<T> {
    private head: Node<T> | null = null;

    push(value: T): void {
        const node = new Node(value);
        if (!this.head) { this.head = node; return; }
        let curr = this.head;
        while (curr.next) curr = curr.next;
        curr.next = node;
    }

    toArray(): T[] {
        const out: T[] = [];
        let curr = this.head;
        while (curr) { out.push(curr.value); curr = curr.next; }
        return out;
    }
}

const list = new LinkedList<number>();
list.push(1); list.push(2); list.push(3);
console.log(list.toArray()); // [1, 2, 3]

```

Challenge 10: Promise.all (Typed)

```

// Type the return of Promise.all when given a mixed-type tuple
async function allTuple<T extends readonly unknown[]>(
    promises: { [K in keyof T]: Promise<T[K]> | T[K] }
): Promise<T> {
    return Promise.all(promises) as Promise<T>;
}

```

```
const [n, s, b] = await allTuple([
  Promise.resolve(1),
  Promise.resolve('hi'),
  true
]);
// n: number, s: string, b: boolean
```

Challenge 11: Memoize

```
function memoize<A extends unknown[], R>(fn: (...args: A) => R): (...args: A) => R {
  const cache = new Map<string, R>();
  return (...args: A): R => {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key)!;
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}

const slowAdd = (a: number, b: number) => { /* slow */ return a + b; };
const fastAdd = memoize(slowAdd);
```

Challenge 12: Validate User from Unknown Data

```
interface User { id: number; name: string; email: string }

function isUser(data: unknown): data is User {
  if (typeof data !== 'object' || data === null) return false;
  const d = data as Record<string, unknown>;
  return (
    typeof d.id === 'number' &&
    typeof d.name === 'string' &&
    typeof d.email === 'string'
  );
}

async function fetchUser(id: number): Promise<User> {
  const data: unknown = await fetch(`/users/${id}`).then(r => r.json());
  if (!isUser(data)) throw new Error('Invalid user payload');
  return data;
}
```

Chapter 23: Behavioral Questions and Final Tips

Common Behavioral Questions (Use the STAR Method)

STAR = Situation, Task, Action, Result. Practice 2-3 stories you can adapt.

- Tell me about a challenging bug you fixed. What was your approach?
- Describe a time you disagreed with a teammate. How did you handle it?
- Walk me through a project you're proud of.
- How do you stay current with new technologies?
- Describe a time you received tough feedback. What did you do?
- Tell me about a time you had to learn something quickly.
- How do you handle tight deadlines and competing priorities?
- Describe a time you simplified a complex problem.

System Design Lite (L2 Bonus Round)

At L2 you may face a lightweight system design question. Common asks: 'Design a URL shortener', 'Design a notification service', 'Sketch a real-time chat app'. What interviewers look for at L2:

- Clarifying questions before designing — scope, users, scale.
- API design — endpoints, request/response shapes (in TypeScript, naturally).
- Data model — entities, relationships, indexes.
- Trade-offs — SQL vs NoSQL, sync vs async, monolith vs microservices.
- Awareness of caching, queues, and basic scaling.

Interview-Day Tactics

- **Think out loud.** Interviewers care about your process, not just the answer.
- **Ask questions.** Clarify constraints before you write a line of code.
- **Start simple.** Get something working, then optimize. Don't pre-optimize.
- **Talk about types.** Show how you'd type your solution — it's a TypeScript interview!
- **Test as you go.** Walk through examples; check edge cases.
- **Own mistakes.** If you spot a bug, say so and fix it — that's a positive signal.
- **Manage time.** If stuck, ask for a hint instead of staying silent.

Red Flags to Avoid

- Using **any** liberally to make errors go away.
- Mutating function parameters or shared state without intent.
- Ignoring null/undefined cases — runtime crashes show poor habits.
- Over-engineering simple problems with deep generics or complex hierarchies.
- Not asking questions before coding.

- Long awkward silences. Narrate; ask for hints.

Questions to Ask the Interviewer

- What does the team's tech stack look like beyond TypeScript?
- How is code reviewed? What does the deployment process look like?
- What does a typical project look like for an L1/L2 engineer here?
- What's the team's approach to testing and code quality?
- What does success look like in the first 90 days?
- What are the biggest challenges facing the team right now?

Appendix — Resources, Cheat Sheets, and Roadmap

Recommended Learning Path (8-12 weeks)

- **Weeks 1-2:** JavaScript fundamentals (variables, scope, closures, promises). Then TypeScript basics (Chapters 1-7).
- **Weeks 3-4:** Type narrowing, OOP, generics, utility types (Chapters 8-11). Build a small CLI tool.
- **Weeks 5-6:** Modules, advanced types, async (Chapters 12-15). Solve type challenges on type-challenges (GitHub).
- **Weeks 7-8:** Build a REST API with Express + TypeScript and a database (Chapters 16, 18). Test it (Chapter 19).
- **Weeks 9-10:** Build a React + TypeScript frontend that consumes your API (Chapter 17).
- **Weeks 11-12:** Interview prep — work through Chapters 20-23 and practice mock interviews.

Books

- **Programming TypeScript** by Boris Cherny — solid foundational read.
- **Effective TypeScript** by Dan Vanderkam — 83 actionable tips, ideal at L2.
- **Learning TypeScript** by Josh Goldberg — friendly modern intro.

Free Online Resources

- **TypeScript Handbook** (typescriptlang.org/docs) — authoritative reference.
- **TypeScript Deep Dive** by Basarat — free comprehensive book online.
- **type-challenges** on GitHub — graded type puzzles, the gold standard.
- **Total TypeScript** by Matt Pocock — short videos, YouTube and blog.

Practice Sites

- LeetCode — language is TypeScript-supported; build problem-solving fluency.
- Codewars — kata style; many TypeScript-tagged challenges.
- Frontend Mentor — UI challenges; rebuild in React + TypeScript.
- Exercism — TypeScript track with mentor feedback.

Cheat Sheet — Most-Used Utility Types

```
Partial<T>           // all properties optional
Required<T>         // all properties required
ReadOnly<T>         // all properties readonly
Pick<T, K>          // keep selected keys
Omit<T, K>           // remove selected keys
Record<K, V>         // object with K keys and V values
Exclude<T, U>        // remove U from T (unions)
Extract<T, U>        // keep U from T (unions)
Nullable<T>         // remove null and undefined
Parameters<F>       // tuple of function param types
```

```
ReturnType<F>    // function return type
Awaited<T>      // unwrap promise type
```

Cheat Sheet — Type Operators

```
keyof T          // union of T's keys
typeof value     // type of a runtime value
T[K]            // indexed access
T extends U ? X : Y // conditional type
infer X         // capture a type in conditional
T & U           // intersection
T | U           // union
as const        // narrow to literal types
satisfies T     // type-check without widening (4.9+)
```

Final Word

TypeScript fluency is built one project at a time. Read code, build things, get comfortable being wrong, and keep iterating. Pay attention to compiler error messages — they're surprisingly precise once you learn to read them.

When you walk into your L1 or L2 interview, remember: the interviewer wants you to succeed. Show your thinking, ask questions, write clear code, and treat types as documentation. Good luck!

— *End of Guide* —